

Dashboard Hydra — Projeto Águas Vivas (IFSC)

Sistema avançado de telemetria e monitoramento ambiental desenvolvido pela equipe de **Engenharia Eletrônica** do Instituto Federal de Santa Catarina (IFSC).

Este arquivo contém o guia completo do sistema, projetado para orientar tanto novos desenvolvedores quanto operadores e mantenedores.

1. Visão Geral do Projeto

O Dashboard Hydra é uma aplicação desenvolvida para receber, armazenar e exibir dados telemétricos de sensores ambientais em tempo real. A comunicação entre o hardware (sensores) e o servidor é feita via protocolo **MQTT**, enquanto a interface de usuário recebe atualizações instantâneas através de **WebSockets**.

2. Tecnologias Envolvidas (Tech Stack)

Frontend (Interface do Usuário)

A aplicação que o usuário acessa no navegador. * **Base:** React (v18.3) com TypeScript. * **Estilização:** Tailwind CSS. * **Bibliotecas de Destaque:** react-leaflet (Mapas), recharts (Gráficos), socket.io-client (Tempo real).

Backend (Servidor)

O “cérebro” invisível do sistema. * **Ambiente:** Node.js com Express (v5.1.0) em TypeScript. * **Comunicação IoT:** mqtt (conectado ao broker Shiftr.io). * **Tempo Real (Frontend):** socket.io.

Banco de Dados

- **Sistema:** PostgreSQL (via Docker).
- **Comunicação:** Prisma ORM.

Infraestrutura

- **Ferramentas:** Docker & Docker Compose para rodar o sistema de forma isolada e segura.
-

3. Arquitetura da Informação (Como Funciona)

1. **Sensores na Água** enviam dados via MQTT (Broker Shiftr.io).
 2. **Servidor (Node.js)** capta essa mensagem e a salva no Banco de Dados.
 3. **Servidor** avisa a Interface do Usuário imediatamente via Socket.IO.
 4. **Interface do Usuário** atualiza os mapas e gráficos sem precisar recarregar a página.
-

4. Guia de Instalação e Uso

Instalação Rápida (Servidor do IFSC)

Este projeto foi dockerizado para garantir que tudo instale automaticamente. Siga estes passos em um computador com Linux limpo:

1. **Copie a pasta completa** do projeto para o computador do servidor no IFSC.
2. Abra o **Terminal** dentro da pasta do projeto e execute o script de automação:

```
chmod +x instalar_hydra.sh  
./instalar_hydra.sh
```

3. Ative as permissões para não precisar deslogar:

```
newgrp docker
```

4. Suba o Sistema com um único comando:

```
docker compose up -d --build
```

Acesso Administrativo e Criação do Primeiro Administrador

Como o banco de dados novo vem zerado por padrão (sem registros na tabela `Usuario`), é necessário cadastrar o primeiro administrador antes de fazer o login.

O projeto possui um script dedicado para a criação do administrador padrão: * **E-mail padrão:** `admin@hydra.local` * **Senha padrão:** `admin123`

Como rodar o script de criação do administrador:

- **No ambiente Docker (Produção/Stack local):** Com a stack rodando, execute o comando a partir do terminal do hospedeiro: `bash docker exec -it hydra_backend npx tsx src/createAdmin.ts` (Ou `docker compose exec backend npx tsx src/createAdmin.ts` a partir do diretório raiz).
- **No ambiente de Desenvolvimento Local (fora de container):** Com as dependências instaladas, vá até a pasta `backend` e execute: `bash cd backend npx tsx src/createAdmin.ts`

Após executar o comando com sucesso, você poderá fazer o login utilizando as credenciais padrão em `http://localhost:3001` (ou na porta correspondente). Recomenda-se alterar a senha imediatamente na aba “Gerenciar Usuários”.

5. Guia para Novos Desenvolvedores (Ambiente Local)

Se você vai modificar o código no seu próprio computador, não precisa rodar tudo pelo Docker.

1. **Suba apenas o Banco de Dados:** `bash docker compose up -d postgres`
2. **Ligue o Backend:**

- Entre na pasta: `cd backend`
- Instale dependências: `npm install`
- Crie as tabelas do banco: `npx prisma db push`
- Ligue: `npm run dev`

3. Ligue o Frontend:

- Abra outro terminal na pasta raiz (Dashboard-PI2-V1).
- Instale: `npm install`
- Ligue: `npm run dev`

Pontos de Atenção

- **Variáveis Secretas (.env):** O backend tem um arquivo `.env` com senhas do banco e do MQTT. Nunca o envie para o GitHub.
 - **MQTT (Shiftr.io):** Se o limite do Shiftr.io for atingido, a comunicação com o hardware vai parar. As chaves de acesso estão no arquivo `.env` ou `docker-compose.yml`.
 - **Banco de Dados:** Mexeu na estrutura de dados (pasta backend/prisma)? Lembre-se de rodar `npx prisma db push` e reiniciar o backend.
-

6. Estrutura e Ingestão de Dados (MQTT & CSV)

O Dashboard Hydra aceita dados telemétricos via protocolo MQTT (em tempo real) ou via importação de arquivo CSV (em lote para histórico).

6.1. Payload MQTT (Tempo Real)

A boia/hardware deve publicar mensagens em formato JSON no tópico Hydra/<boia_id> (onde <boia_id> é o identificador único da boia cadastrada, em letras minúsculas, ex: boia_01).

Estrutura do JSON do Payload:

```
{
  "timestamp": "2026-06-02 16:25",
  "lat": -27.593708,
  "lon": -48.542835,
  "alt": 16.6,
  "tempAr": 25.3,
  "umidAr": 70,
  "pressao": 1012,
  "indiceUV": 5,
  "chuvaAcum": 0,
  "ventoVel": 12,
  "ventoDir": 180,
  "tempAgua": 22.1,
  "phAgua": 7.2,
  "condutivEC": 980,
  "turbidez": 12
}
```

- **Tópico:** Hydra/<boia_id>
 - **Campos de geolocalização (lat, lon, alt):** Opcionais. Se enviados, a posição geográfica da boia será atualizada no mapa.
 - **Campos de Sensores:** Todos os valores numéricos são opcionais no JSON do payload (mas devem ser declarados com o tipo correto). Valores ausentes ou nulos não sobrescreverão as leituras passadas, a menos que especificado.
-

6.2. Estrutura de Importação CSV (Lote)

Na área administrativa de cada boia, é possível fazer o upload de arquivos CSV contendo o histórico de leituras.

- **Separador:** Ponto e vírgula (;).
- **Codificação:** UTF-8.
- **Cabeçalho obrigatório (Primeira Linha):** csv
timestamp;tempAr;umidAr;pressao;indiceUV;chuvaAcum;ventoVel;ventoDir;tempAgua;phAgua;condutivEC;turbidez

Exemplo de conteúdo de arquivo CSV (dados.csv):

```
timestamp;tempAr;umidAr;pressao;indiceUV;chuvaAcum;ventoVel;ventoDir;t  
empAgua;phAgua;condutivEC;turbidez  
2026-05-01 10:00;25.3;70;1012;5;0;12;180;22.1;7.2;980;12  
2026-05-01 10:15;25.1;72;1011.8;4;0;11;175;22.0;7.1;978;13  
2026-05-01 10:30;24.9;75;1011.5;4;0.2;14;190;21.9;7.2;982;12
```

[!IMPORTANT] O formato da data e hora (timestamp) deve seguir o padrão YYYY-MM-DD HH:MM (ex: 2026-05-01 10:00) ou padrão ISO 8601 correspondente.

Valores decimais nos sensores devem utilizar o caractere de ponto (.) como separador decimal.
